UbuCon India 2026

#2026

#UbuConIndia2026

Limiting what AI agents can do

A grant, not a credential

AGNAP: Agent Grant Negotiation and Authorization Protocol

Open source | GNAP · RFC 9635 | Agent runtimes

Daev Mithran · @daevmithran



Canonical Qualcomm penpot



The problem

Agents already do more than asked.

80%

of companies say their AI agents took actions nobody intended.

SailPoint survey, 2025

53%

have had agents go beyond their intended permissions.

Cloud Security Alliance, 2026

23%

have had agents tricked into revealing access credentials.

SailPoint survey, 2025

47%

had a security incident involving an AI agent in the past year.

Cloud Security Alliance, 2026

The key fits the account, not the task.

Real incidents, 2026

The token said yes.

Real incident · April 2026 · Decrypt

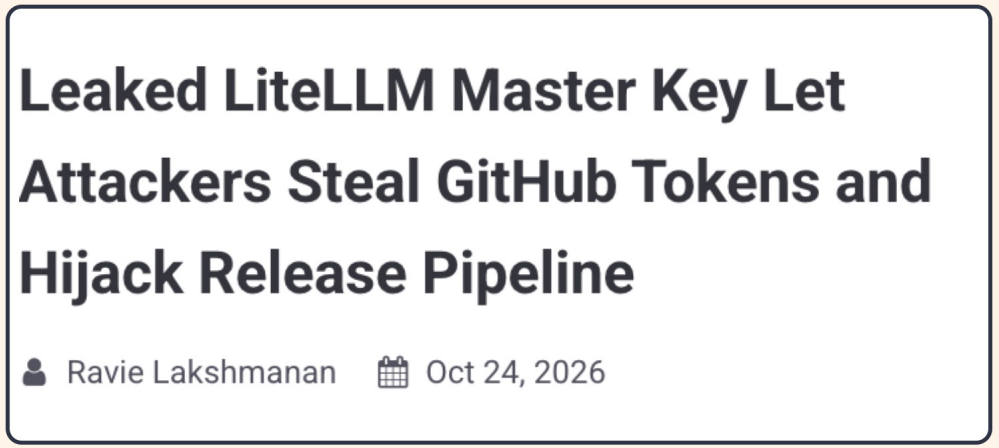


Claude-powered AI agent's confession after deleting a firm's entire database: 'I violated every principle I was given'

PocketOS was left scrambling after a rogue AI agent deleted swaths of code underpinning its business

Deletes. On a staging task, the agent found a token meant for domain management. It reached the whole account: one call wiped production and the backups.

Real incident · October 2026 · The Hacker News



Leaked LiteLLM Master Key Let Attackers Steal GitHub Tokens and Hijack Release Pipeline

Ravie Lakshmanan Oct 24, 2026

Takes over. The agents only needed to open pull requests. Their GitHub token was owner-level, and the leaked key to it was never revoked.

Both tokens were broader than the task. Neither had a "how many", or an end.

It gets worse with subagents

Every handoff loses a little of what you said.

≤1%

One agent

unauthorized actions, working alone

2.7%

Supervisor, depth 1

one level of subagents

19.8%

Supervisor, depth 3

three levels of subagents

No attacker needed. Most constraints are lost at the first handoff.

Source: MasDrift (Xu et al., 2026). 600 benign tasks, 90,000 runs; centralized supervisor trees, pooled across models.

Why tool approval isn’t enough

Runtimes decide by tool name.

Runtime today

tool name → **allow · ask · deny**

A saved “allow” covers every caller, in every task, with any arguments.

With a grant

agent instance + **operation** + **arguments** → **covered or not**

Bound to one running agent, and checked on every call by something it can’t change.

A hook inside the agent’s process is a speed bump, not a wall.

Keeping keys from agents

Three ways to keep keys from agents.

CB4A, Credential Broker for Agents, is an IETF draft (March 2026, expired in September) that names three ways to keep real credentials away from an agent. Weakest first:

1 · Weakest · model C

Hand over the key

The agent holds the real credential. You revoke it later, and hope.

2 · Better · model B

Mint a short-lived token

The agent still holds a usable token. Most APIs can't mint a narrow one, and each subagent gets its own copy.

3 · Strongest · model A

Proxy

The agent never holds a credential. A gateway adds it on the way out. agentgateway, WorkOS Relay and Postern all chose this.

Even a proxy needs a grant: in 2025 an injected issue made the GitHub MCP server, which held the key, leak private repos.

The idea

A grant, not a credential.

Each running agent gets its own grant for one task, checked outside the agent.

One running agent

bound to that agent's own key

Typed operations

exact actions and argument limits

Ceilings

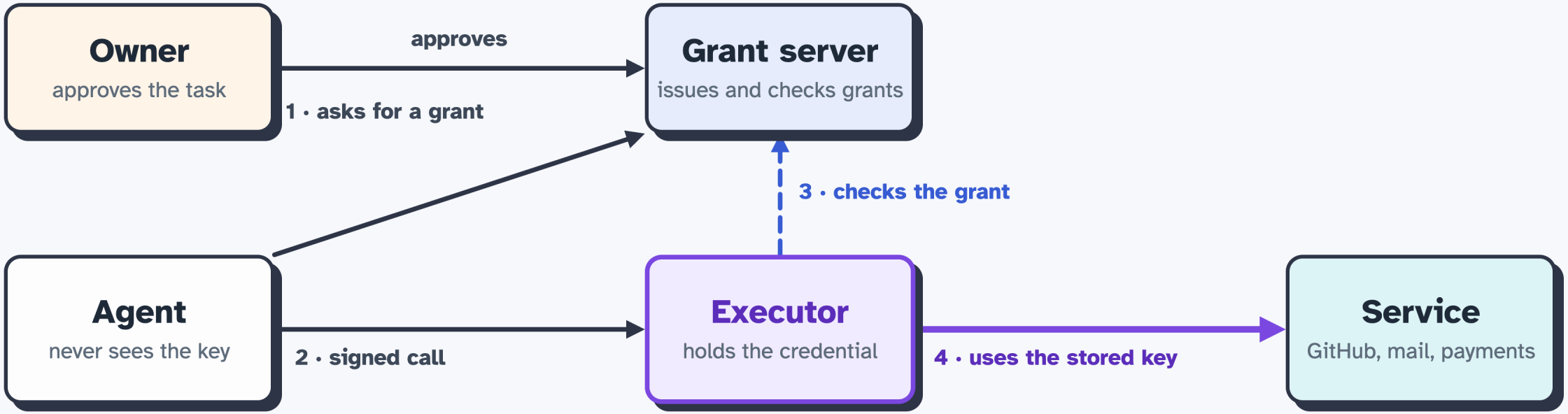
expiry, call count, spend

Revocable

cancel the task and its calls stop

How it works · Model A, plus a grant for each task

The agent asks. Something else holds the key.

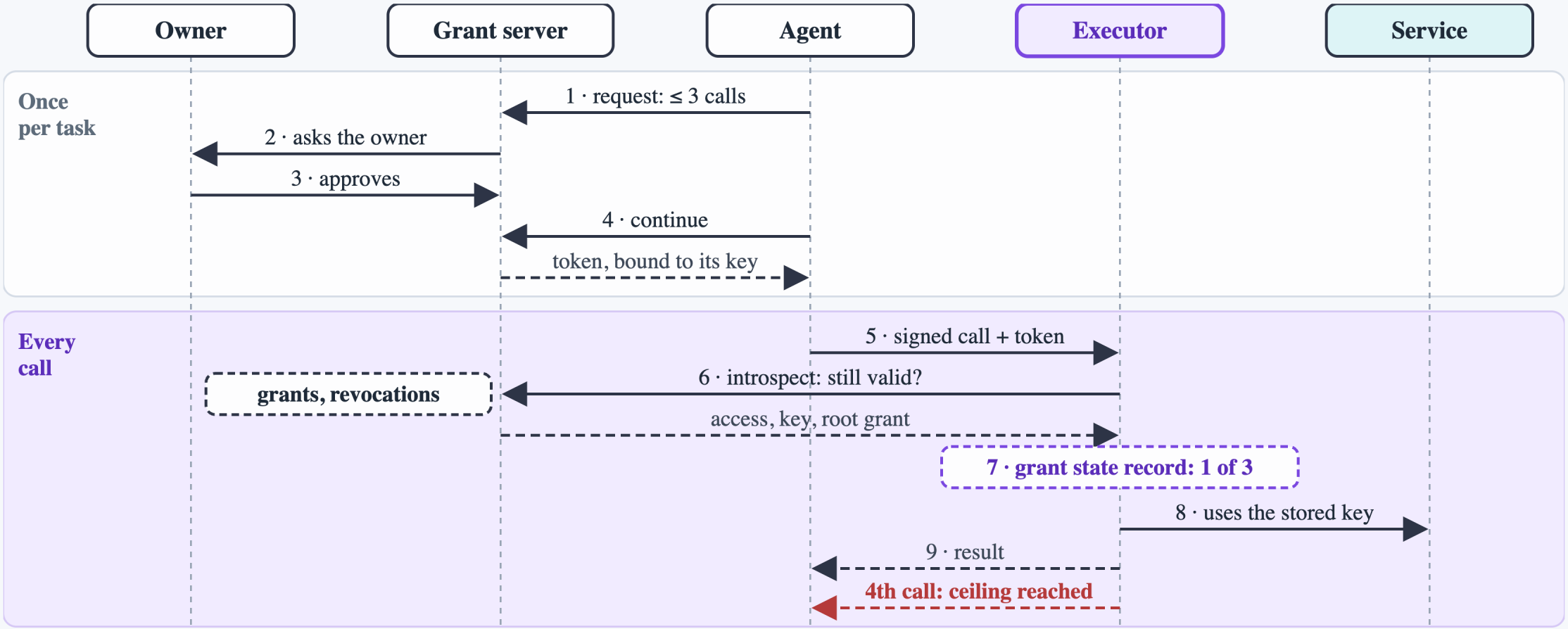


The agent proves who it is. It never holds the downstream credential.

What CB4A leaves out, the grant adds: argument constraints, a shared ceiling, the agent tree.

How it works · GNAP, step by step

Get a grant once. Every call is checked.



Grants and revocations live in the grant server. The grant state record lives in the executor.

Pluggable roles

Who can play each part?

Grant server · decides

GNAP (RFC 9635) AGNAP uses this

Key-bound tokens, rich access, wait and continue.

OAuth 2 + DPoP with extensions

Add RAR, introspection and a root-grant claim. No continue step.

AAuth with extensions

Strong agent identity and approvals. Needs a live check and a way to say exactly what's allowed.

Executor · holds the key, checks every call

AGNAP executor built

Checks the grant on every call and keeps the grant state record.

agentgateway could be

An MCP and A2A proxy with external authorization: plug the grant check in.

An MCP gateway could be

Docker's MCP Gateway already keeps the tools' secrets away from the agent.

CLI tools on Ubuntu designed

A systemd service as its own user holds the keys; AppArmor keeps the agent out.

A grant server only has to answer: still valid? What's allowed? Which key? Which root grant?

Why GNAP

In GNAP, the grant is the protocol.

What an agent needs	OAuth 2	AAuth	GNAP
Token bound to the agent's key	add-on (DPoP)	✓	✓
Operations, arguments and ceilings in the request	add-on (RAR)	—	✓
Pause for the owner's approval, then resume	add-on (CIBA)	✓	✓
Narrow or extend a live grant	—	partly	✓
How many specs	five or six RFCs	one draft	one RFC

Any service, any credential. Only the executor holds it; GitHub never adopts GNAP.

Already in production for payments: Interledger Open Payments runs GNAP.

What a grant says

Operations and arguments, not scopes.

```
{
  "type": "agent_operation",
  "operations": {
    "github.issues.list": {
      "repository": { "exact": "acme/docs" },
      "state": { "wildcard": true }
    }
  }
}
```

Covered

List open issues in acme/docs

Denied

List issues in acme/secrets: a different repository

Asks the owner

Post a comment: not in this grant yet

Closed world: an unknown operation or argument is denied.

Delegation **Design · specified, not built yet**

Every agent gets its own key. Authority only narrows.

Root agent · key R
grant approved by the owner

Child A · key A
grant inside the root's

Grandchild A1 · key A1
grant inside A's

Child B · key B
grant inside the root's

1 · The child makes its own key
Only the public half ever leaves it.

2 · The parent vouches for it
"This key may do this part of what I may do." The runtime confirms it spawned the child.

3 · The child gets its own grant
Bound to its key, and only if it fits inside the parent's.

No key is copied down the tree, and no child can hold more than its parent.

Subagents

Children share the parent’s ceiling. They don’t copy it.

Copy the parent’s token

Root task · 10 comments

10 10 10

each child checks only its own copy

Up to 30 comments go out

One grant state record, kept by the executor

Root task · 10 comments

4 posted 4 posted 2 refused

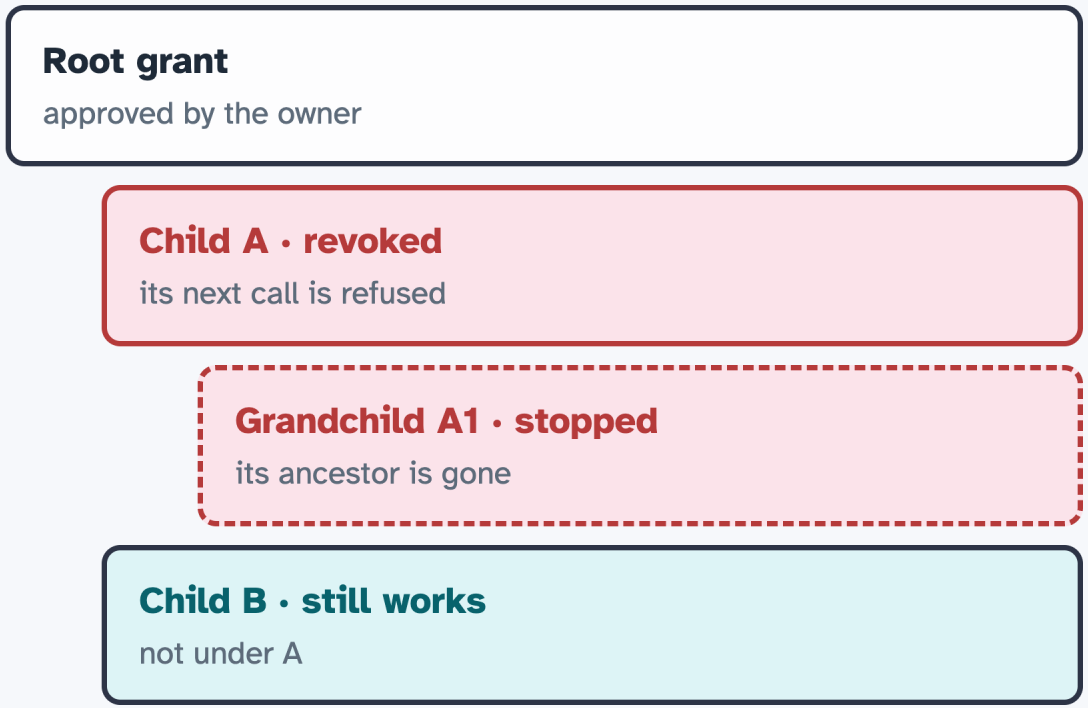
the third posts 2, then the record is at 10

Never more than 10

The runtime reports each spawn, so the tree is the real one, not the app’s description of it.

Revocation **Built: refused on the next call · Not yet: owner revoke, cascade**

Revoke one grant, and its subtree stops.



1 · Revoke at the grant server

The owner, or the agent for its own grant. No parent signature needed.

2 · It cascades down

Descendants are marked revoked, and every check also looks up the ancestors.

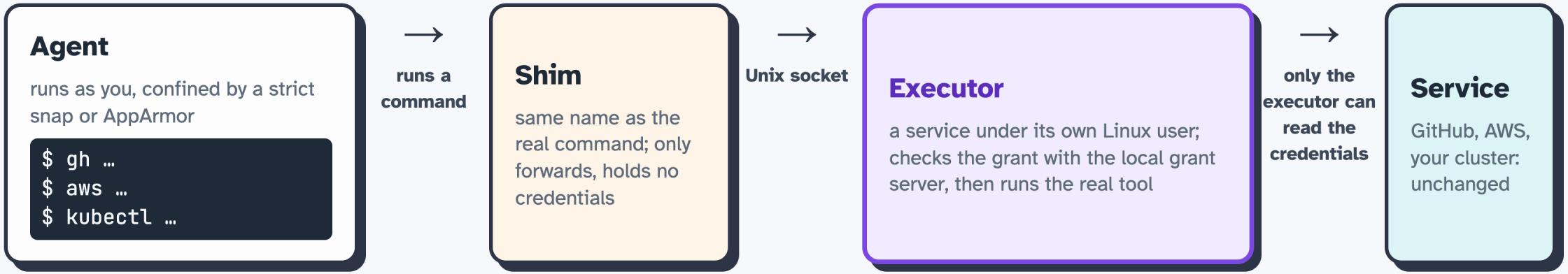
3 · The executor refuses

It checks every call, with no cache. Nothing reaches the service.

Receipts prove where a grant came from. Only the live check proves it still holds.

On your Linux box **Design · CLI shims not built yet**

The same design, on your laptop.



What's a shim?

A small stand-in with the same name as a real command, earlier on $PATH. It passes the arguments to the executor and prints the answer. Like the stand-ins snap puts in /snap/bin.

Who plays which part

Executor: holds the credentials, runs the tool. **Grant server**, on localhost: issues and checks grants. **You**, in your terminal: the owner who approves. The shim is none of these.

Skipping the shim gets nothing

The agent can still run the real tool, but has no credentials for it. Only the executor's user can read them.

Like sudo, but for one running agent, one task, with a ceiling and an expiry.

What you can do today

Treat an agent like an untrusted process.

Keep tokens out of reach

Put them behind a proxy the agent can’t read, such as agentgateway or Postern.

Confine the process

Run agents in a container, a strictly confined snap, or under an AppArmor profile.

Narrow the key behind the proxy

The proxy uses a least-privilege key per agent, short-lived where the service allows. Not one admin token for everything.

Judge effects, not words

Log and test what happened downstream, not what the model said.

A prompt is not a boundary unless every effect passes through it.

Thank you

A grant, not a credential

Give each agent a task, not your keys.

AGNAP on GitHub github.com/DaevMithran/agnap

GNAP, RFC 9635 rfc-editor.org/rfc/rfc9635

Twitter / X @daevmithran

Built: grant server, executor, grant state record, owner CLI, DeepSeek Harness plugin. Next: child grants, owner revoke, CLI shims.

Open source | GNAP · RFC 9635 | Agent runtimes

Ubucon India 2026

Not alone

Everyone covers some of it. So does AGNAP.

Approach	Agent never holds the service credential	Constraints on each argument	Ceiling shared across siblings	Agent tree from the runtime	Verified agent identity
AAuth (draft -11)	—	—	—	—	✓
AAT / Tenuo (draft -02)	—	✓	—	—	partly
CB4A (expired draft)	partly	—	—	—	✓
Transaction Tokens for Agents	—	—	—	—	partly
Caracal	✓	—	—	—	partly
Amazon AgentCore	✓	✓	partly	—	✓
Bounded Agents (APC)	—	—	partly	—	✓
AGNAP	✓	✓	✓	✓	partly

✓ in the spec or docs **partly** covers part of it — not found in public specs or docs · November 2026

Where it stands

Early, open source, and measured honestly.

Built

- GNAP grant server (RFC 9635, RFC 9767) and RFC 9421 signatures
- Executor (agnapd): holds the credentials
- Grant state record with call ceilings
- Owner CLI: decisions signed with the owner's key
- DeepSeek Harness plugin, tested in the real Harness

Next

- Child grants from real spawn events
- Owner revocation of a live grant
- Benchmarks: MasDrift, AgentDojo with Ajar, Bounded Agents scenarios

Contributions, and hard questions, are welcome.